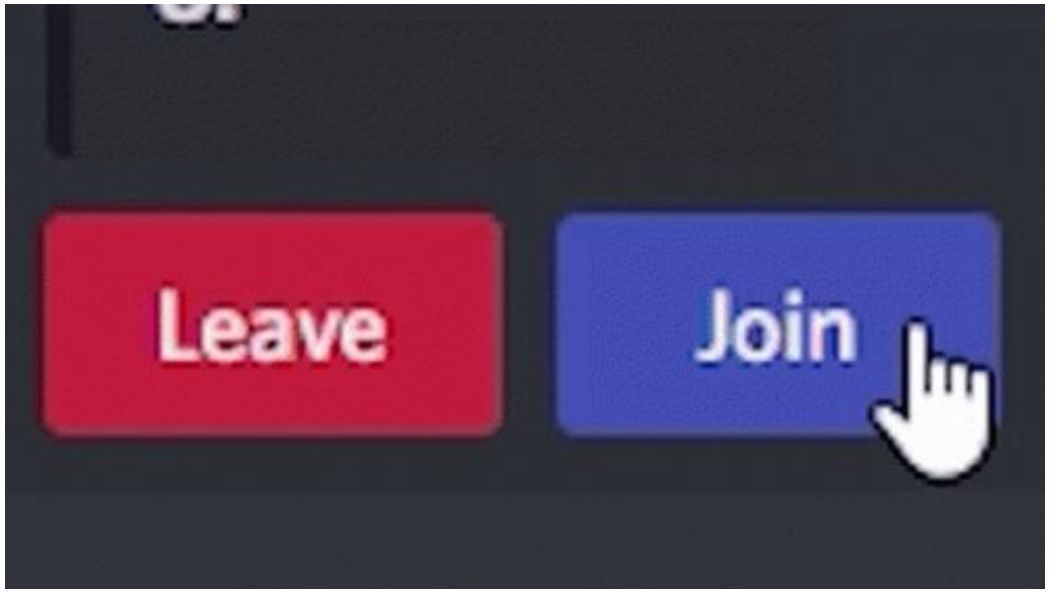
Newton School of Technology

12

Commercial APIs

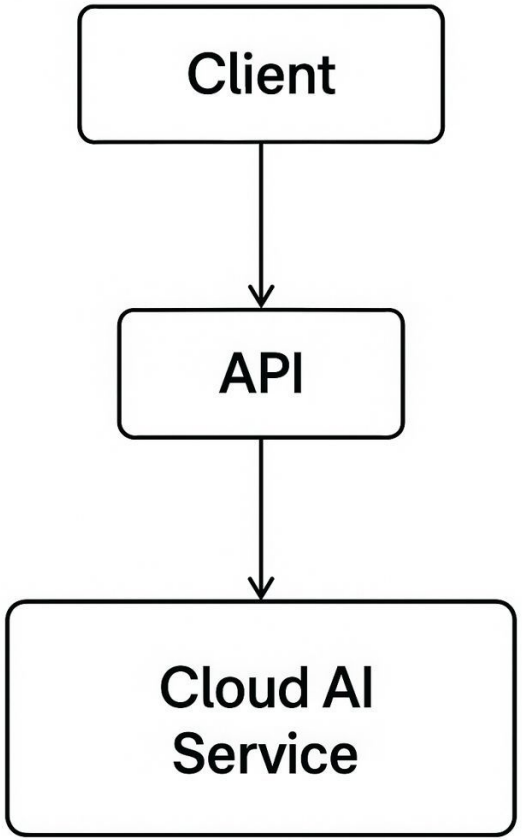
Join the Lecture on NS Portal :)

Newton School of Technology



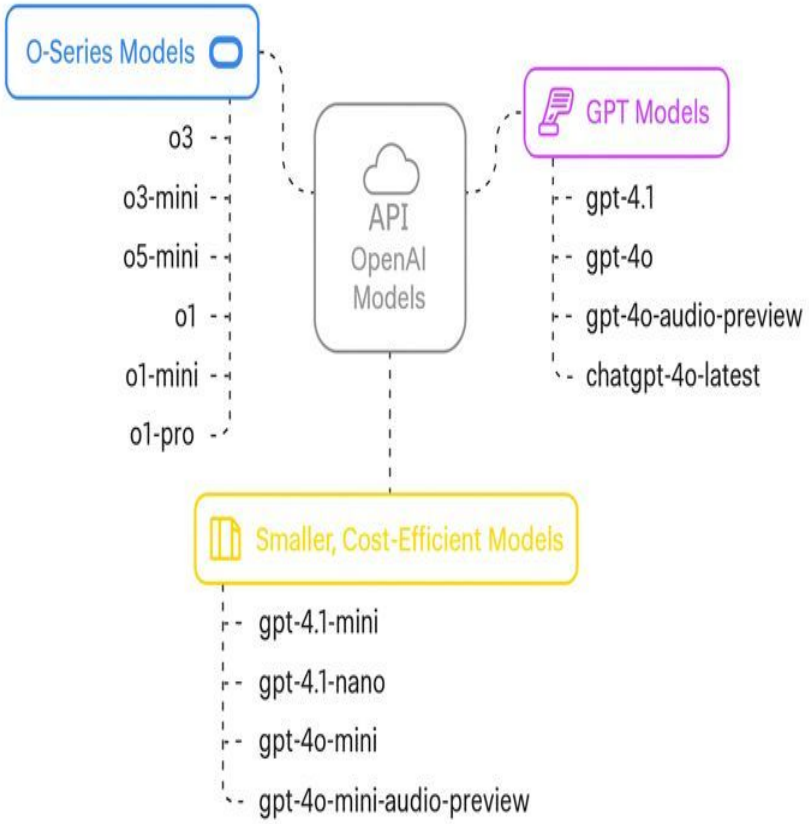
Introduction to Commercial APIs

- Commercial APIs provide secure and reliable access to cloud-based services.
- In AI systems, they allow the use of powerful models without local training or hosting.
- Complex infrastructure like GPUs and scaling is managed by the provider.
- Services are offered on a subscription or pay-as-you-go pricing model.
- These APIs are designed for enterprise-level security and scalability.
- They are commonly used in chatbots, document processing, and automation.



What is the OpenAI API?

- OpenAI API gives access to advanced AI models using simple HTTP requests.
- It supports tasks like text generation, summarization, translation, and embeddings.
- The API uses a REST-based design with JSON requests and responses.
- All models are hosted and maintained by OpenAI, removing infrastructure overhead.
- Secure authentication is handled through API keys.
- Multiple model options are available for different use cases and costs.



Core Components of OpenAI API

Newton School of Technology

Component	Purpose
Client App	Sends API requests
API Gateway	Auth & routing
AI Models	Generate responses
Usage Tracker	Billing & limits
Response Handler	JSON output

Which component is responsible for authentication in the OpenAI API?

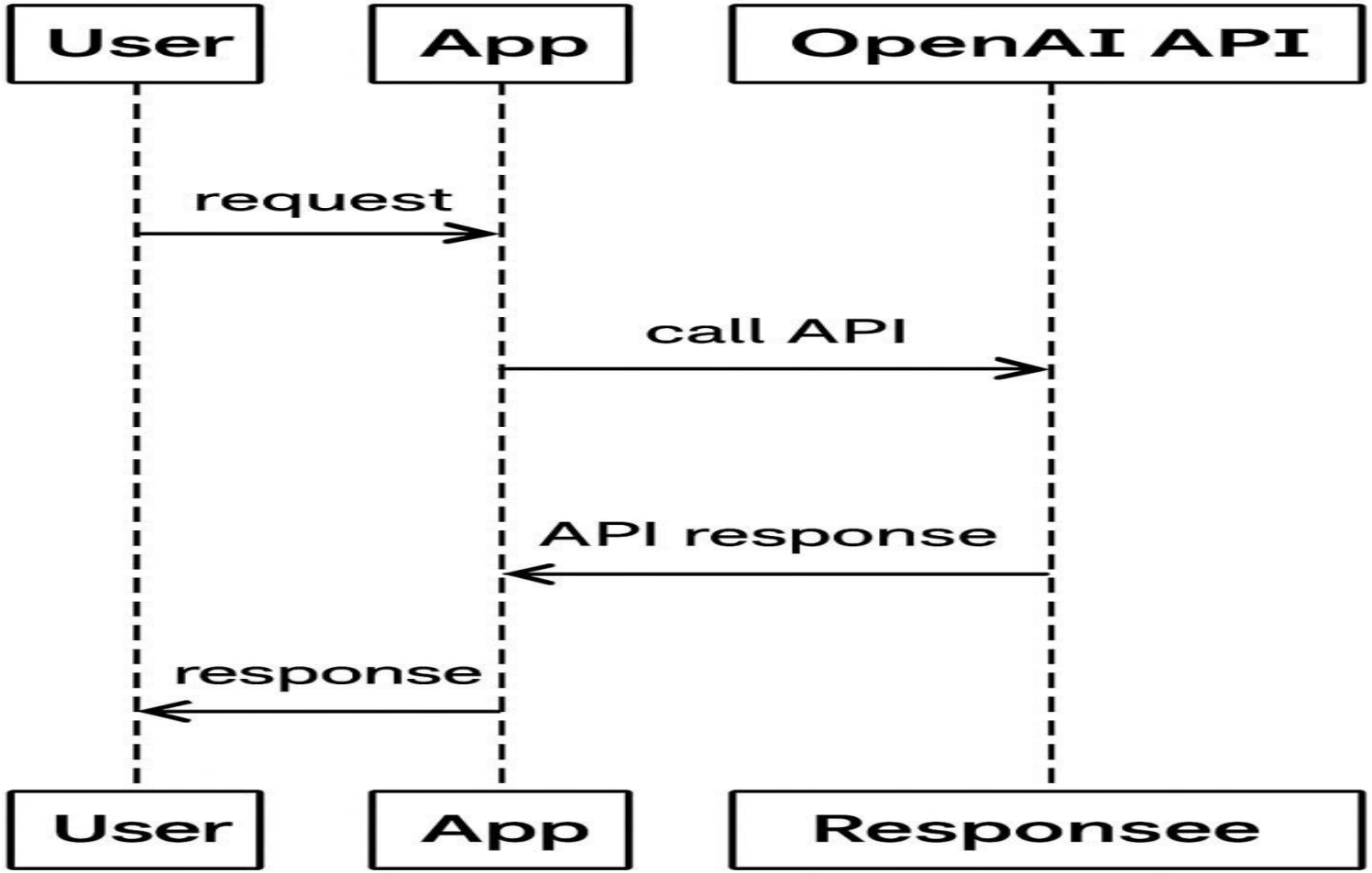
OpenAI Models and Capabilities

Newton School of Technology

- OpenAI offers multiple models designed for different workloads.
- Text-generation models are used for chatbots, content creation, and reasoning.
- Embedding models convert text into vectors for search and retrieval.
- Some models are optimized for lower cost and faster responses.
- Other models focus on higher reasoning accuracy and complex tasks.
- Choosing the right model affects cost, latency, and output quality.

Model Type	Primary Use
Chat Models	Conversations, Q&A
Embedding Models	Search, RAG
Lightweight Models	Low-cost tasks
Advanced Models	Complex reasoning

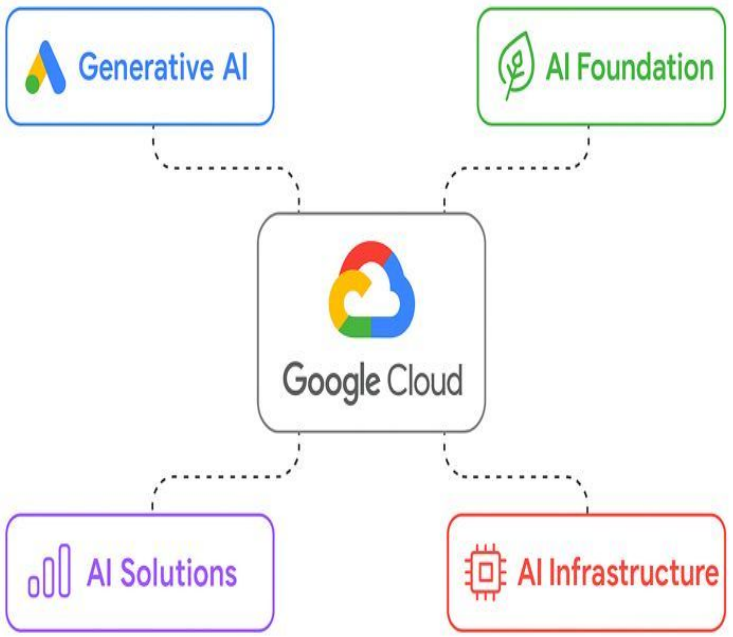
Typical OpenAI API Request Flow



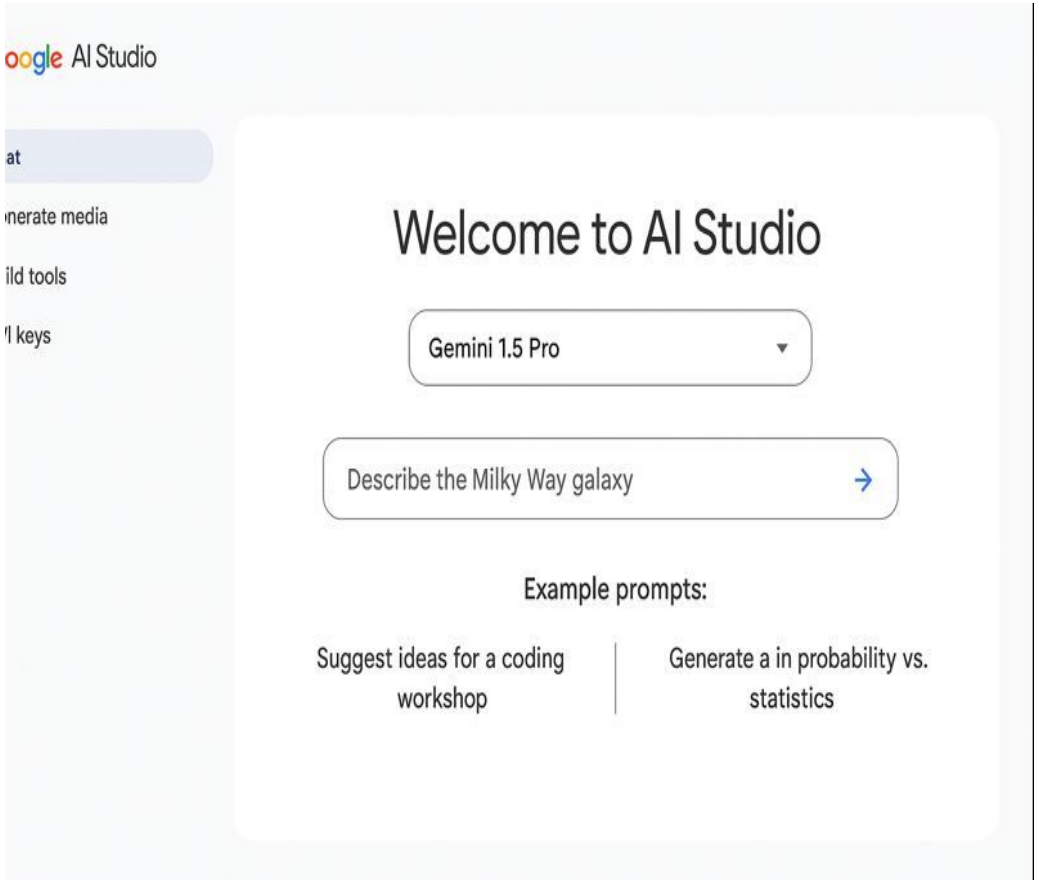
What problem does the OpenAI API solve for developers?

Introduction to Google AI APIs

- Google AI APIs provide access to advanced machine learning models developed by Google.
- They are part of the Google Cloud AI ecosystem, including Vertex AI and Gemini.
- These APIs support natural language, generation, and multimodal AI capabilities.
- They are built for large-scale, enterprise-grade applications.
- Google AI APIs integrate tightly with services like BigQuery and Cloud Storage.
- Strong focus is placed on security, compliance, and data governance.



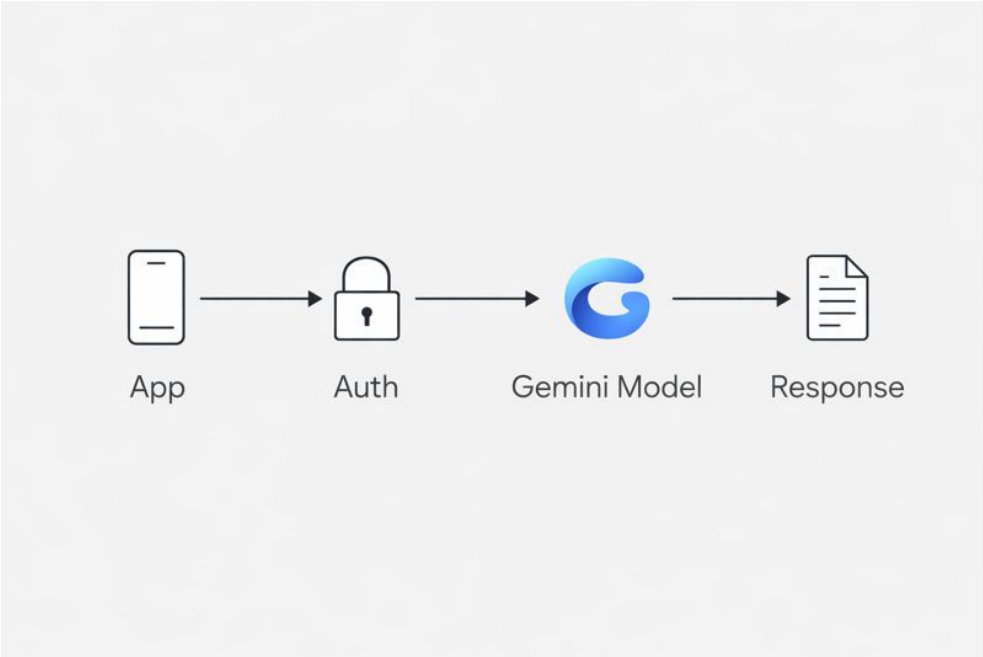
What is the Gemini API?



- Gemini is Google's AI model family.
- It supports text, reasoning, and code tasks.
- Some models also work with images.
- Gemini is available via Google AI Studio and Vertex AI.
- APIs can be used through REST or SDKs.
- Built for enterprise performance and scale.

Google AI API Architecture

Component	Description
Client App	Sends AI requests
API Gateway	Auth & routing
Gemini Models	AI inference
TPU Infrastructure	High performance
Monitoring Layer	Logging & billing



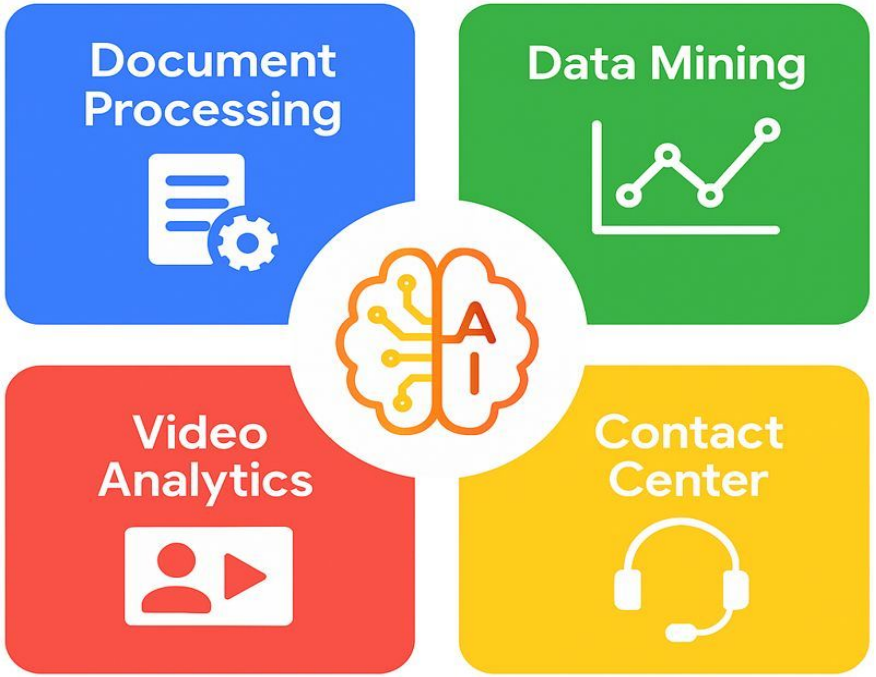
Google AI vs OpenAI

Newton School of Technology

Aspect	OpenAI API	Google AI API
Primary Focus	Simple APIs and fast developer onboarding	Enterprise integration and cloud-native workflows
Platform Style	Standalone AI API platform	Part of Google Cloud ecosystem
AI Capabilities	Advanced generative AI models	Advanced generative and multimodal AI models
Billing Model	Token-based pricing	Integrated with Google Cloud billing
Cost Management	OpenAI usage dashboard	Cloud budgets, quotas, and alerts
Model Access	Simple model selection	Managed via Vertex AI / Gemini
Best Suited For	Startups and rapid development	Large-scale enterprise applications

Use Cases of Google AI APIs

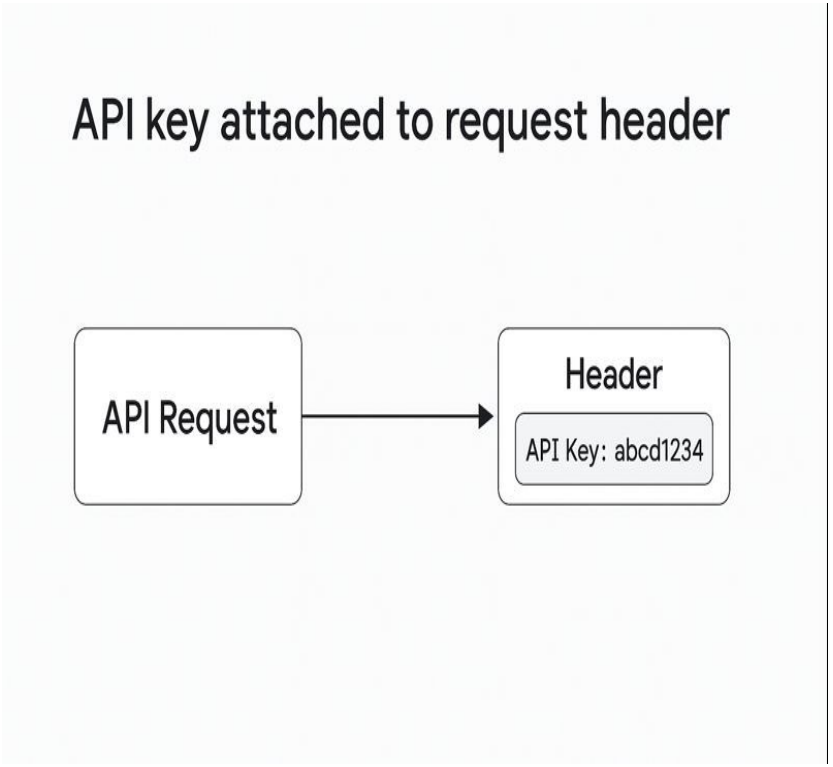
- Intelligent chatbots integrated with Google Cloud services.
- Automated document analysis and summarization.
- Customer support systems with multilingual capabilities.
- Data-driven insights using AI + BigQuery.
- Content moderation and classification pipelines.
- Code generation and developer productivity tools.
- AI-powered search and recommendation systems.



How does Google AI integrate with other cloud services?

Understanding API Keys in Commercial APIs

- An API key is a unique identifier used to authenticate API requests.
- It tells the provider who is making the request and what access is allowed.
- API keys are used to track usage, billing, and rate limits.
- Every AI API request requires a valid API key.
- API keys are sensitive credentials and must be kept secure.
- Leaked API keys can cause unauthorized access and high costs.



API Key Lifecycle Management

Newton School of Technology

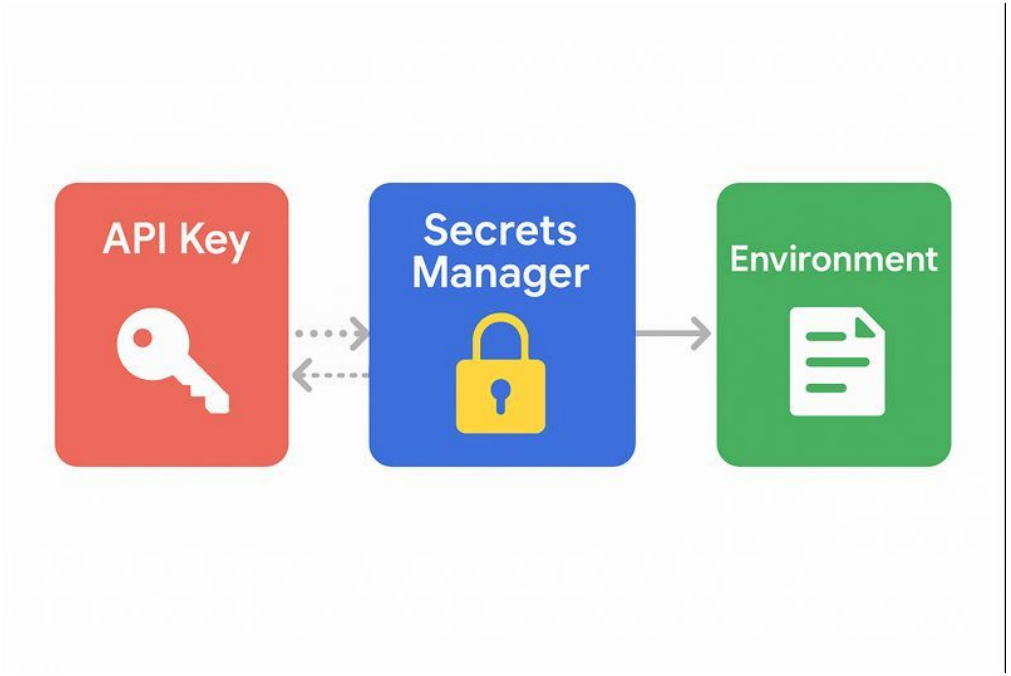
- API keys are first **generated** in the provider's dashboard.
- They are then **configured** with permissions or usage restrictions.
- Applications **store and use** the key to authenticate API calls.
- Over time, keys may need **rotation** to reduce security risks.
- Compromised or unused keys should be **revoked immediately**.
- Monitoring helps detect abnormal usage patterns.
- A well-managed lifecycle minimizes security vulnerabilities.

Stage	Description
Creation	Generate key
Configuration	Set permissions
Usage	API authentication
Rotation	Periodic renewal
Revocation	Disable compromised keys

Why should API keys never be hardcoded in source code?

Best Practices for Storing API Keys

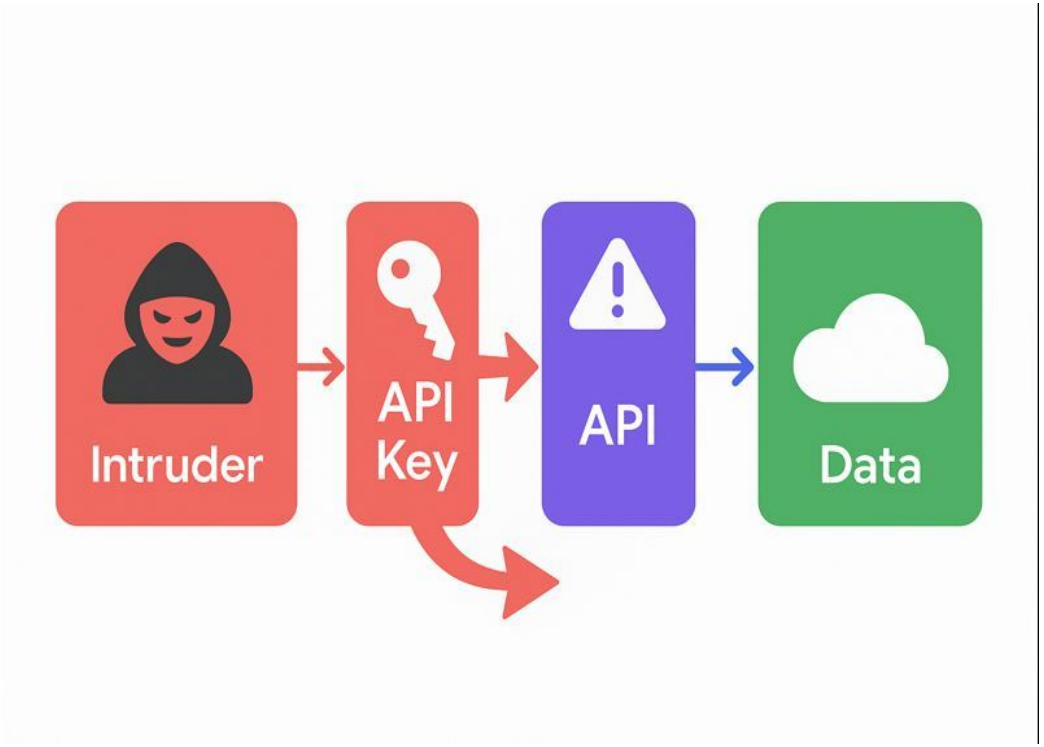
- API keys should never be hardcoded in source code.
- Environment variables are commonly used for secure storage.
- Secrets managers provide stronger protection for sensitive keys.
- Frontend applications must not expose API keys.
- Backend servers should securely handle API requests.
- Access should be limited by role and environment.



Why is key rotation important?

Common API Key Security Risks

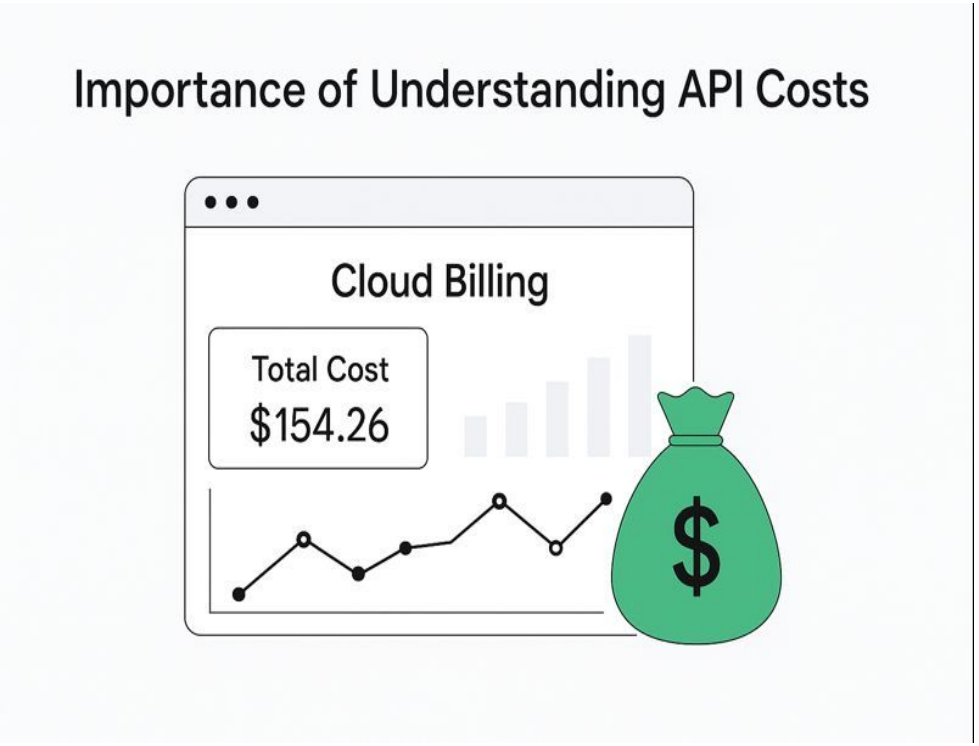
- Exposed keys in public repositories can be exploited instantly.
- Unauthorized usage can cause unexpected billing spikes.
- Attackers can misuse keys to generate harmful or illegal content.
- Lack of usage monitoring delays detection of breaches.
- Shared keys across teams increase accountability issues.
- No rotation policy increases long-term security risks.
- Proper governance is required to mitigate these threats.



What happens if an API key is leaked publicly?

Importance of Understanding API Costs

- Commercial AI APIs use a pay-as-you-go pricing model.
- Costs depend on API usage and data processed.
- Usage is measured using tokens, requests, or compute time.
- Poor cost planning can lead to high or unexpected bills.
- Cost awareness helps choose the right model for each task.
- Clear billing is essential for production and enterprise systems.



Example -Calling an AI API

```python
from openai import OpenAI
import os

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

response = client.responses.create(
    model="gpt-4.1-mini",
    input="Explain refund policy in simple words"
)

print(response.output_text)
```

Customer Support Bot

1. User asks a question
2. Backend calls AI API
3. AI generates response
4. Response shown to user

Why do we keep this code in backend and not in frontend?

OpenAI API Pricing Model

- OpenAI primarily charges based on **number of tokens processed**.
- Tokens include both **input tokens** (prompt) and **output tokens** (response).
- Different models have different per-token pricing.
- Lightweight models are cheaper but less capable.
- Advanced models cost more but provide better reasoning.
- Pricing is usage-based, with no fixed monthly fee by default.
- Billing details are available in the OpenAI dashboard.

Factor	Description
Input Tokens	Prompt size
Output Tokens	Response size
Model Type	Affects cost
Usage Volume	Total billing

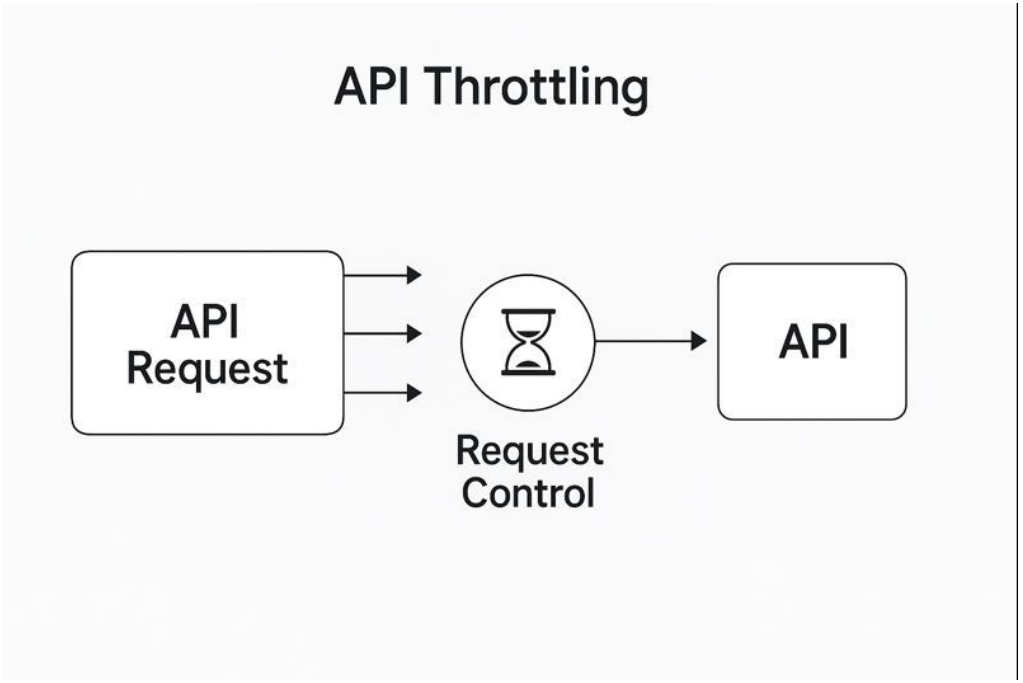
Billing Structure of Google AI APIs

- Google AI APIs are billed through **Google Cloud Billing**.
- Charges may be based on requests, tokens, or compute usage.
- Gemini usage is often linked with **Vertex AI pricing**.
- Billing accounts allow budget limits and alerts.
- Costs integrate with other Google Cloud services.
- Enterprises benefit from consolidated cloud billing.
- Detailed cost breakdowns are available per project.

Aspect	Description
Billing Platform	Google Cloud
Pricing Metric	Tokens / Requests
Budget Alerts	Supported
Enterprise Billing	Integrated

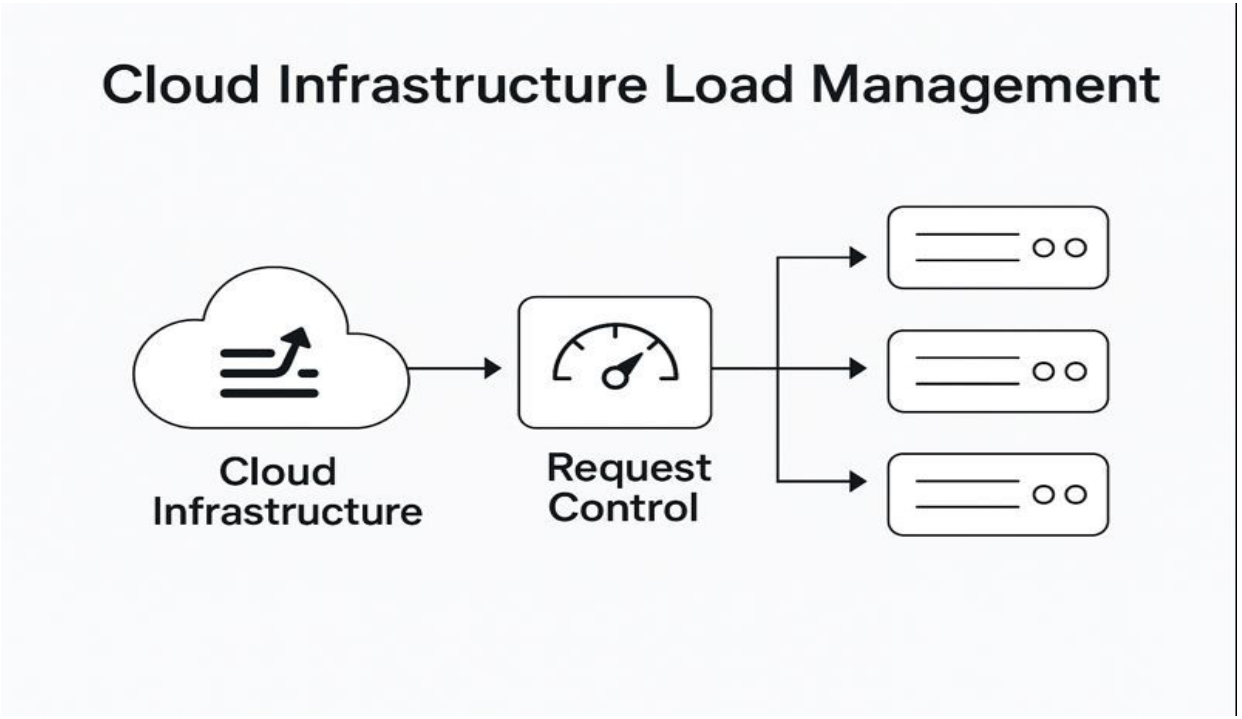
Understanding Rate Limits in APIs

- Rate limits control how many API requests are allowed in a time period.
- They protect systems from overload and misuse.
- Limits ensure fair usage for all users.
- Restrictions may apply per API key, user, or IP address.
- Exceeding limits temporarily blocks requests.
- Understanding rate limits helps build reliable applications.



Purpose of Rate Limiting in AI Systems

- AI inference requires significant compute resources.
- Rate limits prevent system overload during traffic spikes.
- They help maintain low latency for all users.
- Rate limiting discourages malicious or automated abuse.
- Providers use limits to enforce pricing tiers.
- Higher plans often offer higher rate limits.
- Rate limits improve system stability and reliability.



Why do AI APIs enforce rate limits?

Understanding Rate Limit Errors (HTTP 429)

Newton School of Technology

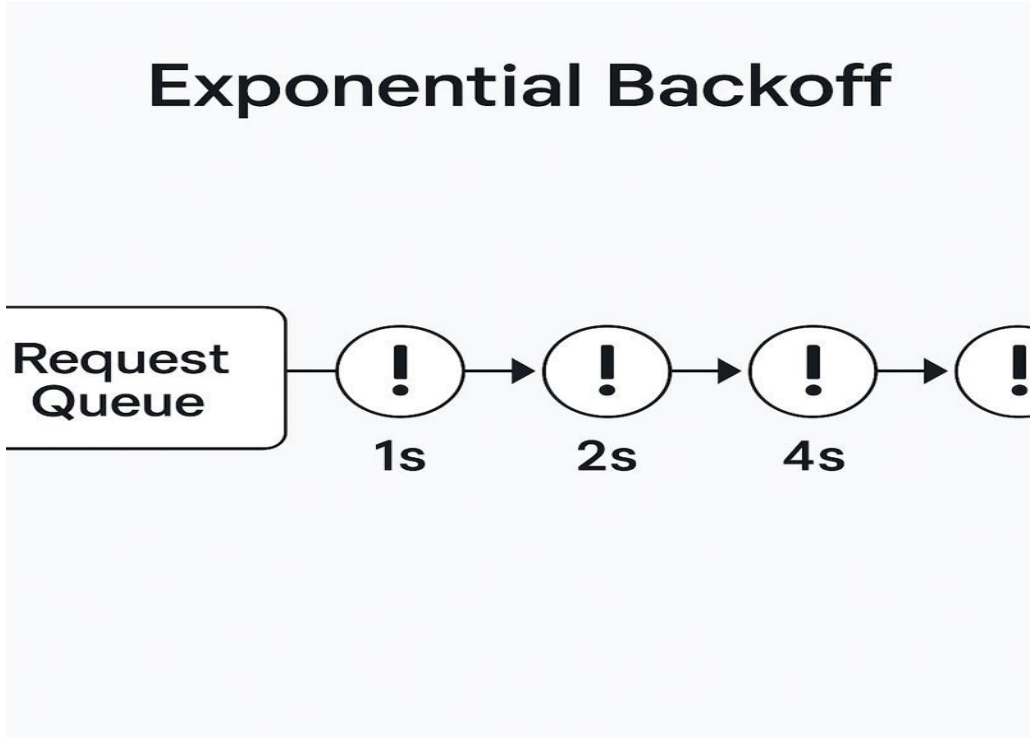
- When rate limits are exceeded, APIs return an HTTP 429 error.
- The error indicates too many requests in a short time.
- Responses may include retry-after headers.
- Ignoring rate limit errors can cause repeated failures.
- Proper handling improves application resilience.
- Logs and monitoring help detect frequent limit hits.
- Developers should design retry strategies.

Error Code	Meaning
429	Too Many Requests
403	Access restricted
401	Auth issue

What is the purpose of HTTP 429 errors?

Handling Rate Limits Gracefully

- Implement exponential backoff for retries.
- Queue requests instead of sending bursts.
- Cache results to reduce duplicate calls.
- Monitor API usage metrics continuously.
- Upgrade plans if higher throughput is required.
- Separate workloads across multiple keys if allowed.
- Design systems to degrade gracefully.



How does exponential backoff help applications?

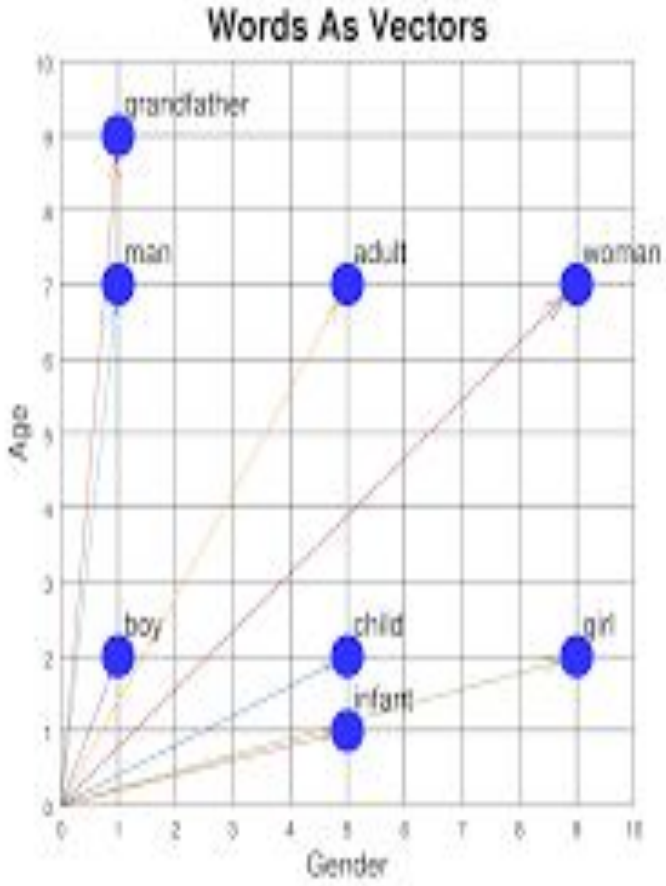
Rate Limits – OpenAI vs Google AI

- OpenAI enforces per-minute and per-day limits.
- Google AI applies quotas through Google Cloud.
- Limits depend on account tier and region.
- Google provides quota management tools.
- OpenAI provides usage dashboards and limits.
- Both platforms allow limit increase requests.
- Proper planning avoids service disruptions.

Feature	OpenAI	Google AI
Limit Type	Requests/Tokens	Quotas
Monitoring	Dashboard	Cloud Console
Increase Request	Supported	Supported
Enforcement	Automatic	Automatic

Introduction to Word Embeddings

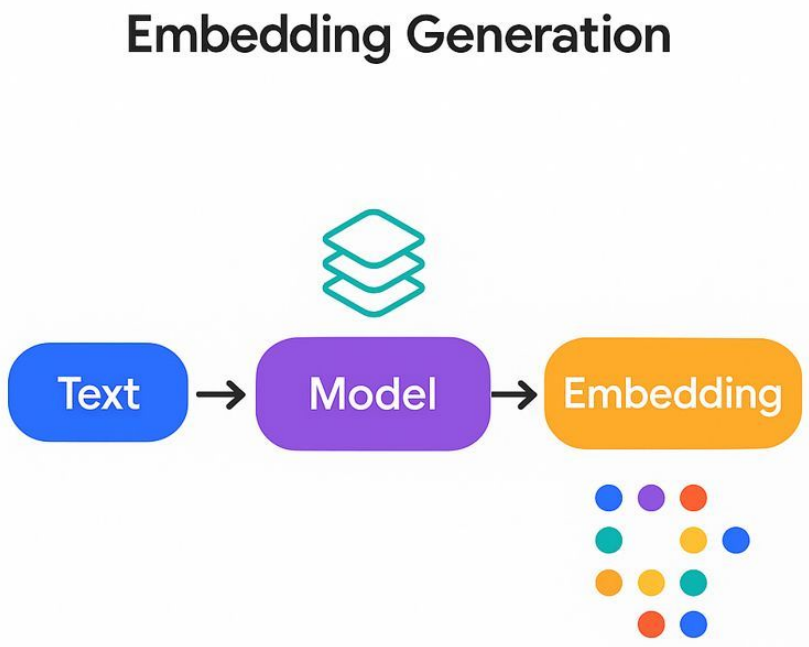
- Word embeddings are numerical representations of text.
- They convert words or sentences into vectors of numbers.
- These vectors capture semantic meaning and context.
- Similar words have vectors that are close in space.
- Embeddings enable machines to “understand” language mathematically.
- They are foundational to search, recommendation, and RAG systems.
- Modern AI systems rely heavily on embeddings for text understanding



Why are embeddings better than keyword search?

Working Mechanism of Embeddings

- Text is tokenized into words or subwords.
- Each token is mapped to a high-dimensional vector.
- The model learns relationships from large text datasets.
- Contextual similarity is encoded in vector distances.
- Mathematical operations measure similarity and relevance.
- Embeddings are generated once and reused efficiently.
- This approach is faster than generating text repeatedly.



How do embeddings capture semantic meaning?

Example- Document Search Using Embeddings

```python
from openai import OpenAI
import numpy as np

client = OpenAI()

note = "75% attendance is mandatory for exams"
question = "Exam attendance requirement?"

note_vec = client.embeddings.create(
    model="text-embedding-3-small",
    input=note
).data[0].embedding

question_vec = client.embeddings.create(
    model="text-embedding-3-small",
    input=question
).data[0].embedding

similarity = np.dot(note_vec, question_vec)
print(similarity)
```

Searching a Document(Note)

1. Note→ Embeddings
2. Question → Embedding
3. Compare similarity
4. Return best match

Student Notes → Embeddings → Similarity Match → Answer

```
Similarity score: High → Relevant match
```

Embedding APIs in Commercial Platforms

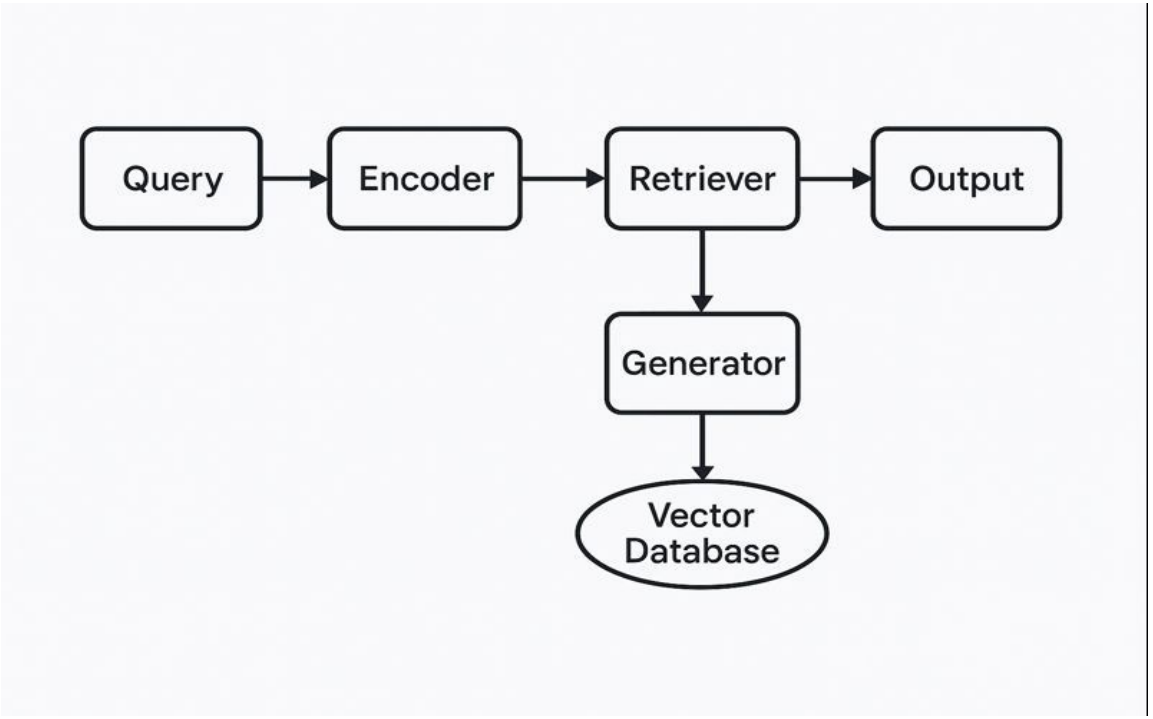
Newton School of Technology

- OpenAI provides dedicated embedding models via API.
- Google AI offers embeddings through Vertex AI.
- Both generate fixed-length vector outputs.
- Embeddings are stored in vector databases.
- They are used for similarity search and clustering.
- Model choice impacts accuracy and cost.
- APIs return embeddings in JSON format.

Feature	OpenAI	Google AI
Output	Vector	Vector
Use Cases	RAG, search	RAG, analytics
Integration	Simple API	Cloud-native
Cost Model	Token-based	Cloud billing

Use Cases of Word Embeddings

- Semantic search in documents and knowledge bases.
- Retrieval-Augmented Generation (RAG) pipelines.
- Recommendation systems based on similarity.
- Clustering of documents or customer feedback.
- Duplicate detection and plagiarism checking.
- Question-answering systems.
- Context-aware chatbots.



Task

Newton School of Technology

You are building a Student Help Portal for a college.

The portal should:

1. **Answer general student questions using an AI API**
2. **Search study notes using embeddings**
3. **Use APIs securely and efficiently**

Key Learnings on Commercial AI APIs

Newton School of Technology

- Commercial AI APIs provide scalable AI capabilities.
- OpenAI and Google AI are leading providers.
- API key management is critical for security.
- Cost awareness prevents unexpected billing.
- Rate limits ensure system stability.
- Word embeddings enable semantic understanding.
- Choosing the right API depends on use case and scale.

Newton School of Technology

Please fill the feedback form.